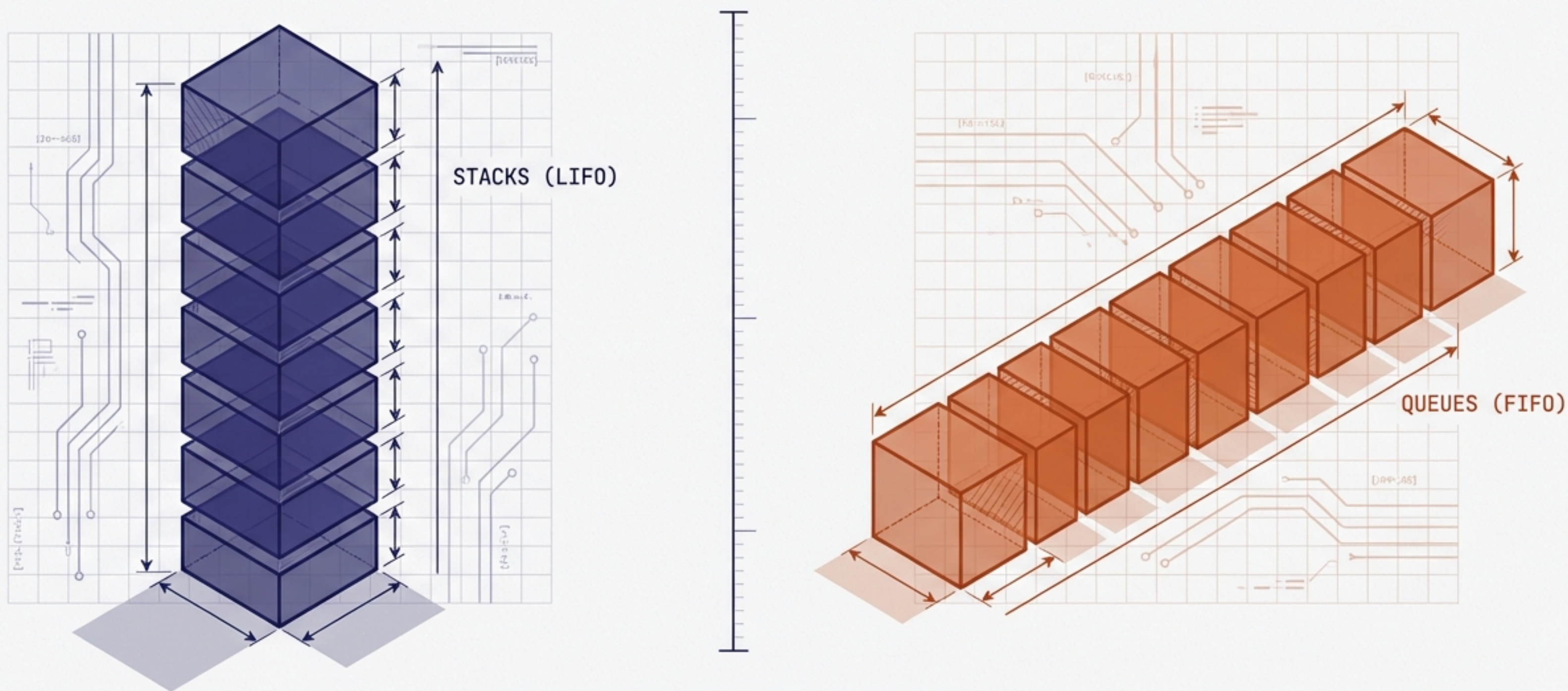
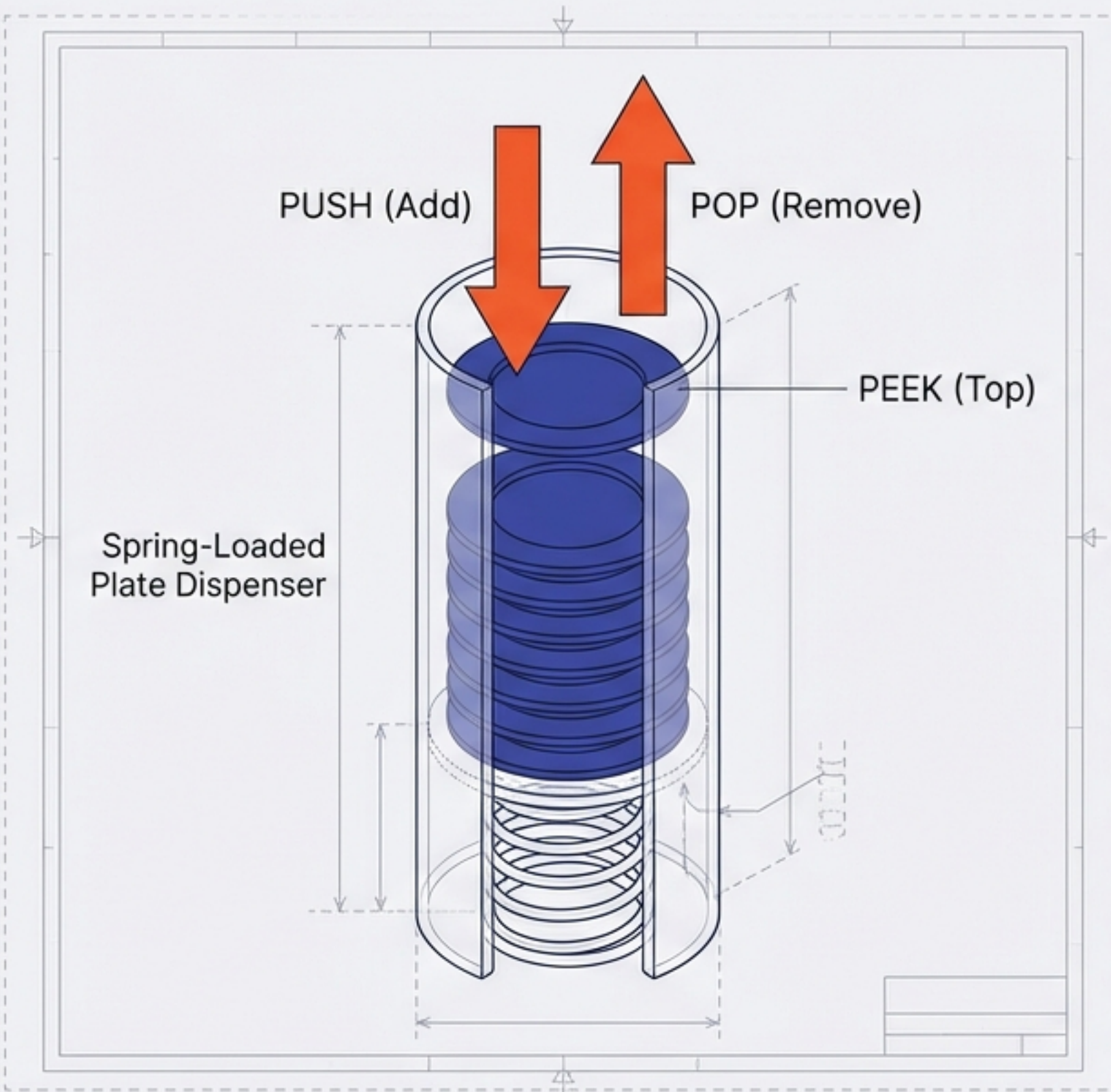


The Logic of Order: Stacks, Queues, and the Baseball Game

Advanced DSA & LeetCode Bootcamp Breakdown



The Stack Refresher: Vertical Logic



Core Concept: LIFO

Last In, First Out. The last element added is the first one removed.

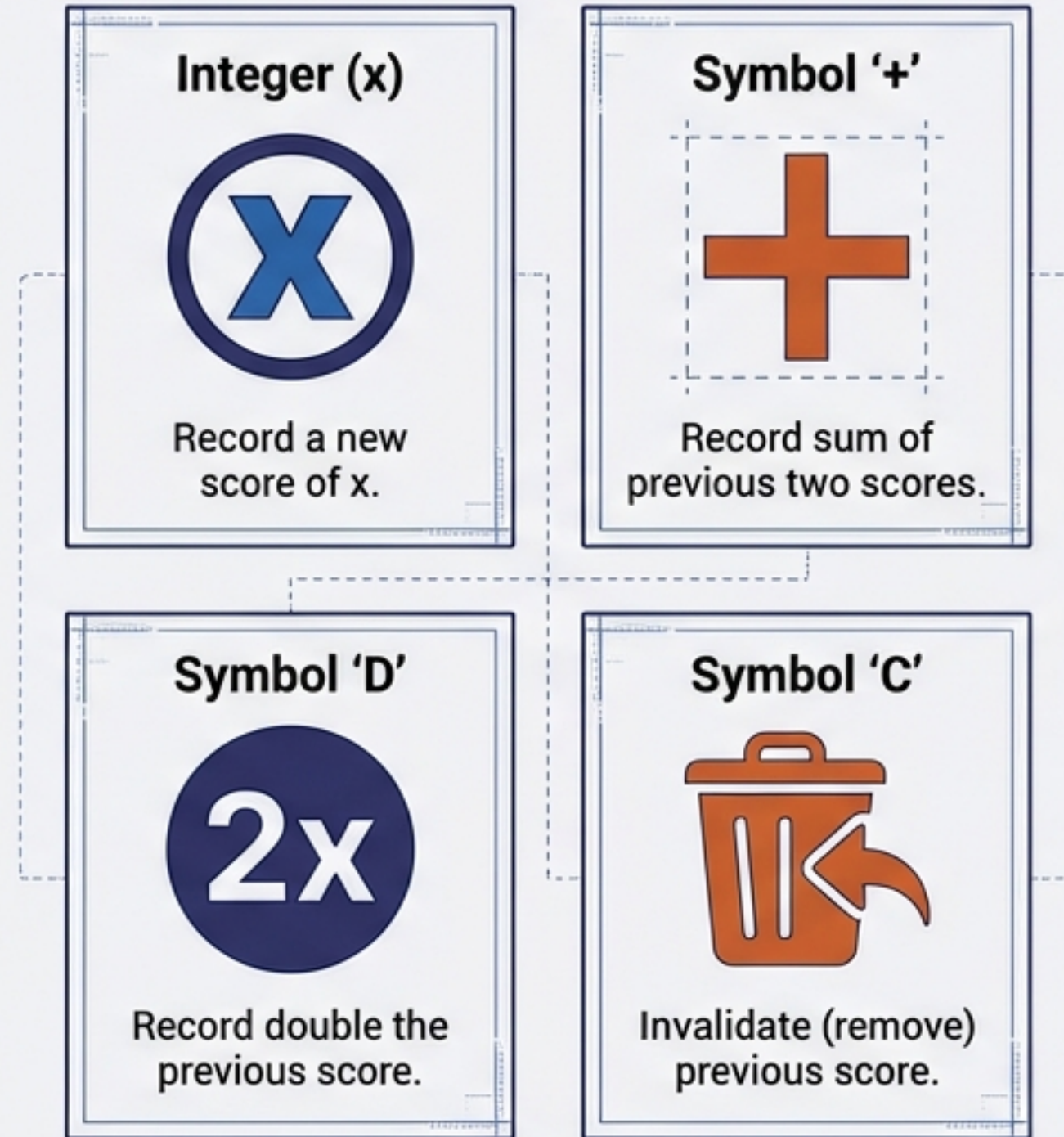
Key Operations

- Push: Add an element to the top.
- 2. Pop: Remove the top element.
- 3. Peek/Top: View the top element (Index -1).
- IsEmpty: Check if the stack has no elements.

Correction: It is 'Last In First Out', not 'First In Last Out'.

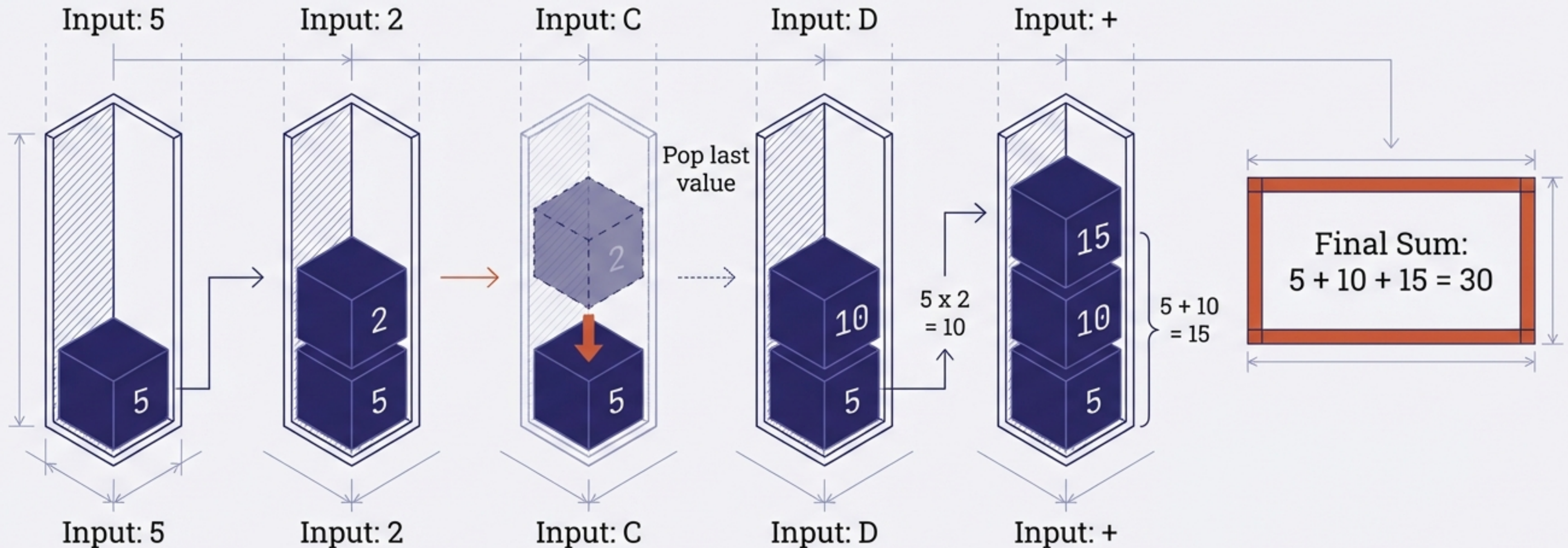
The Challenge: The Baseball Game (LeetCode 682)

You are the scorekeeper. The rules are dynamic.



Visualizing the Algorithm: A Dry Run

Input Sequence: [5, 2, C, D, +]



The Implementation (Python)

```
stack = []  
for op in operations:  
    if op == "+":  
        stack.append(stack[-1] + stack[-2])  
    elif op == "D":  
        stack.append(2 * stack[-1])  
    elif op == "C":  
        stack.pop()  
    else:  
        stack.append(int(op))  
return sum(stack)
```

Handles "C" by removing the last valid score.

Peeking at top two elements without removing them.

Convert string input to integer before pushing.

Navigating Constraints & Edge Cases

Q: What if the stack is empty when '+' arrives?

The problem guarantees **Valid Operations**. Constraints ensure that for '+', there will always be two previous scores, and for 'C' or 'D', at least one.



Q: What does the '32-bit Integer' warning mean?

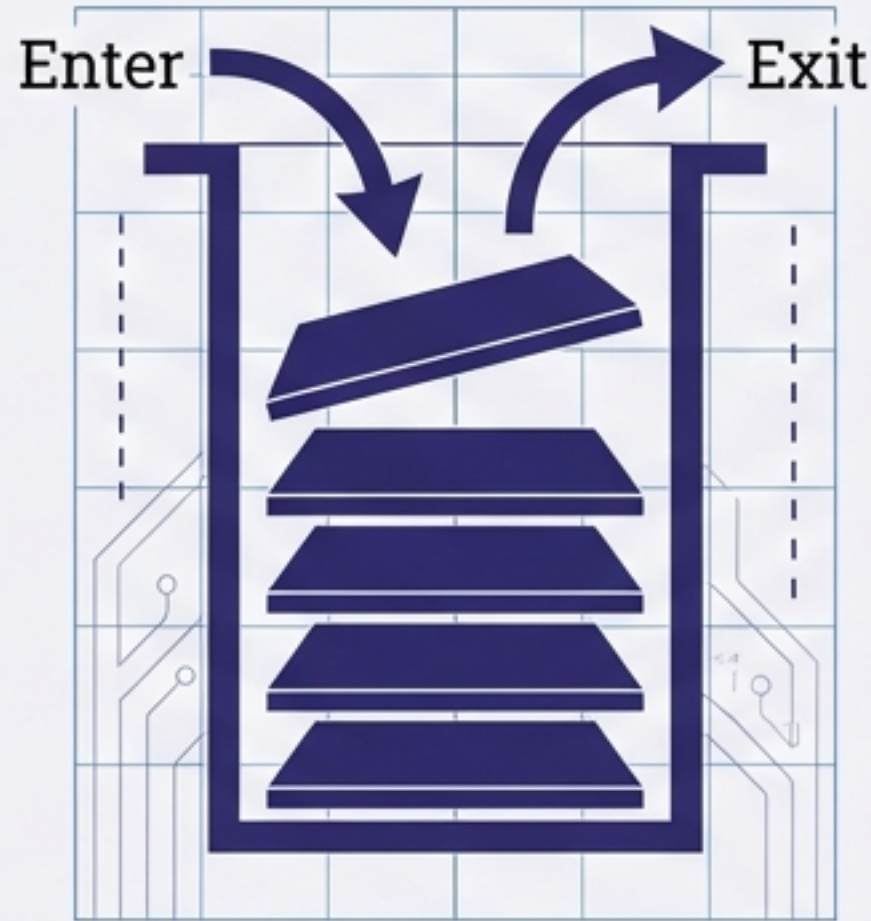
This is a system limitation warning. Intermediate calculations (like doubling) will not exceed standard memory limits.

"It's about space complexity... ensuring the solution fits on a standard minimum system."



Shifting Perspective: From LIFO to FIFO

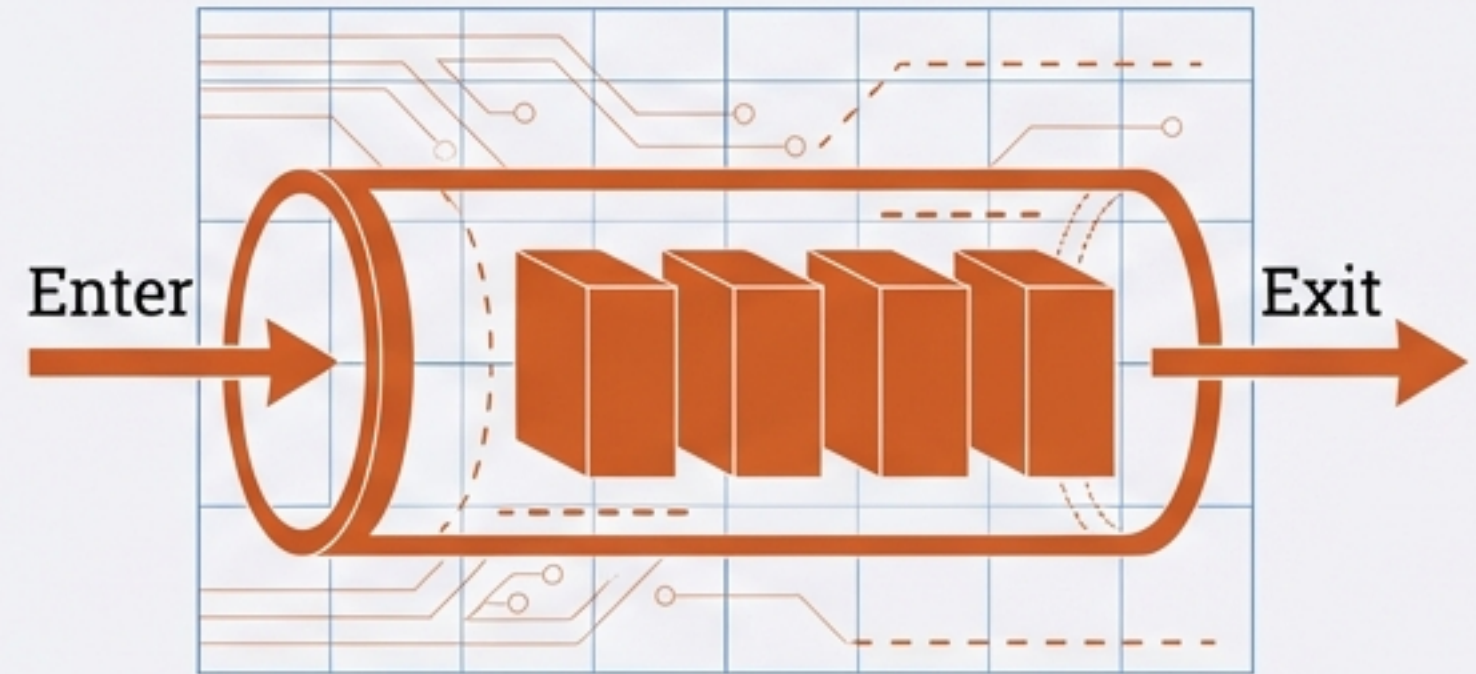
LIFO (Last In, First Out)



Metaphor: Plate Dispenser

Focus: Top (-1)

FIFO (First In, First Out)



Metaphor: ATM Line

Focus: Front (0)

The Language of Queues

Core Operations

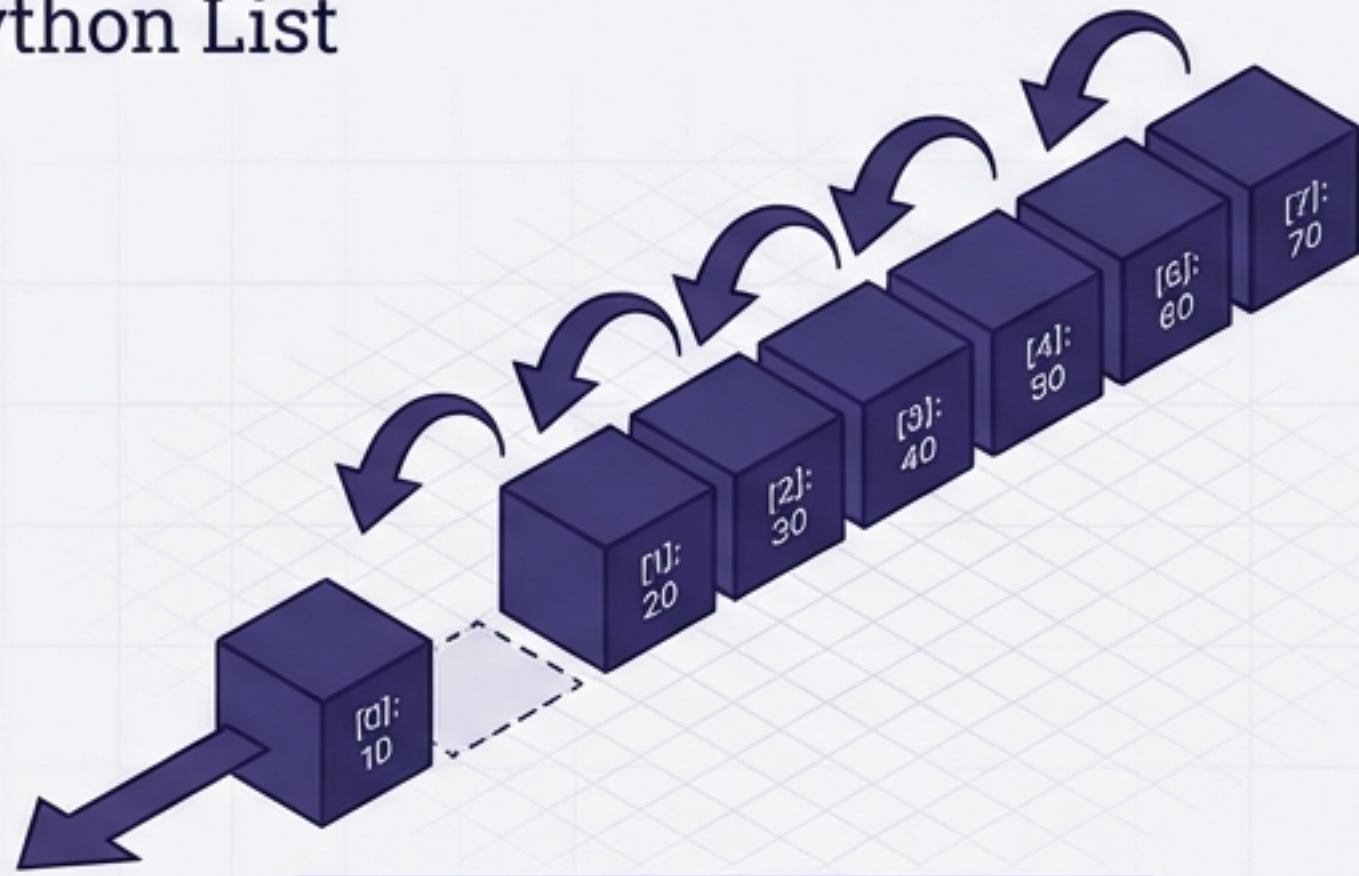
1. **Enqueue**: Add an item to the **Rear**.
2. **Dequeue**: Remove an item from the **Front**.
3. **Peek**: View the front element.

Action	Stack Term	Queue Term
Add	Push	Enqueue
Remove	Pop	Dequeue
View	Peek / Top	Peek / Front

Optimizing for Efficiency: `list` vs. `deque`

The Problem

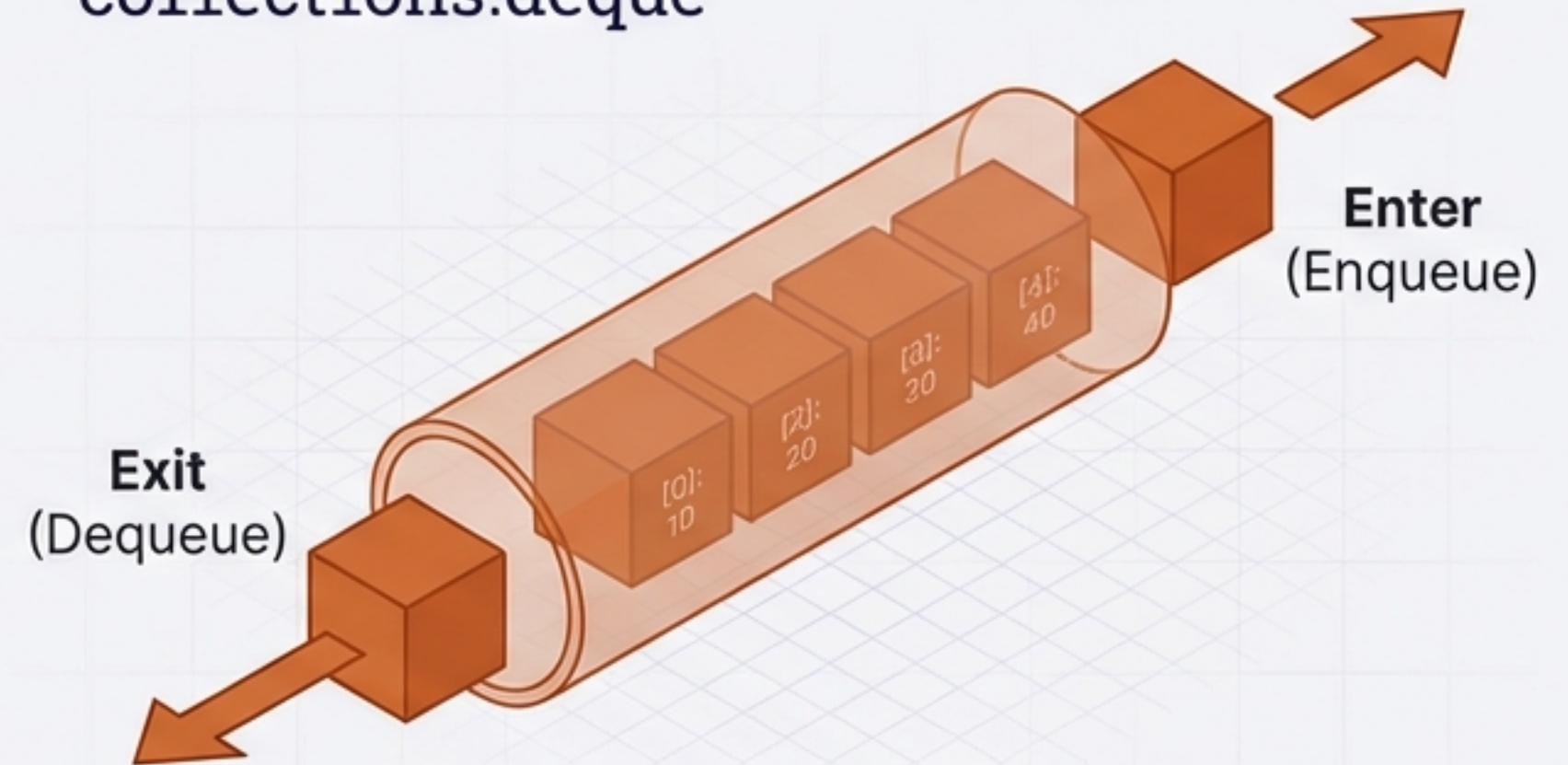
Python List



Time Complexity: $O(n)$ - Slow

The Solution

`collections.deque`



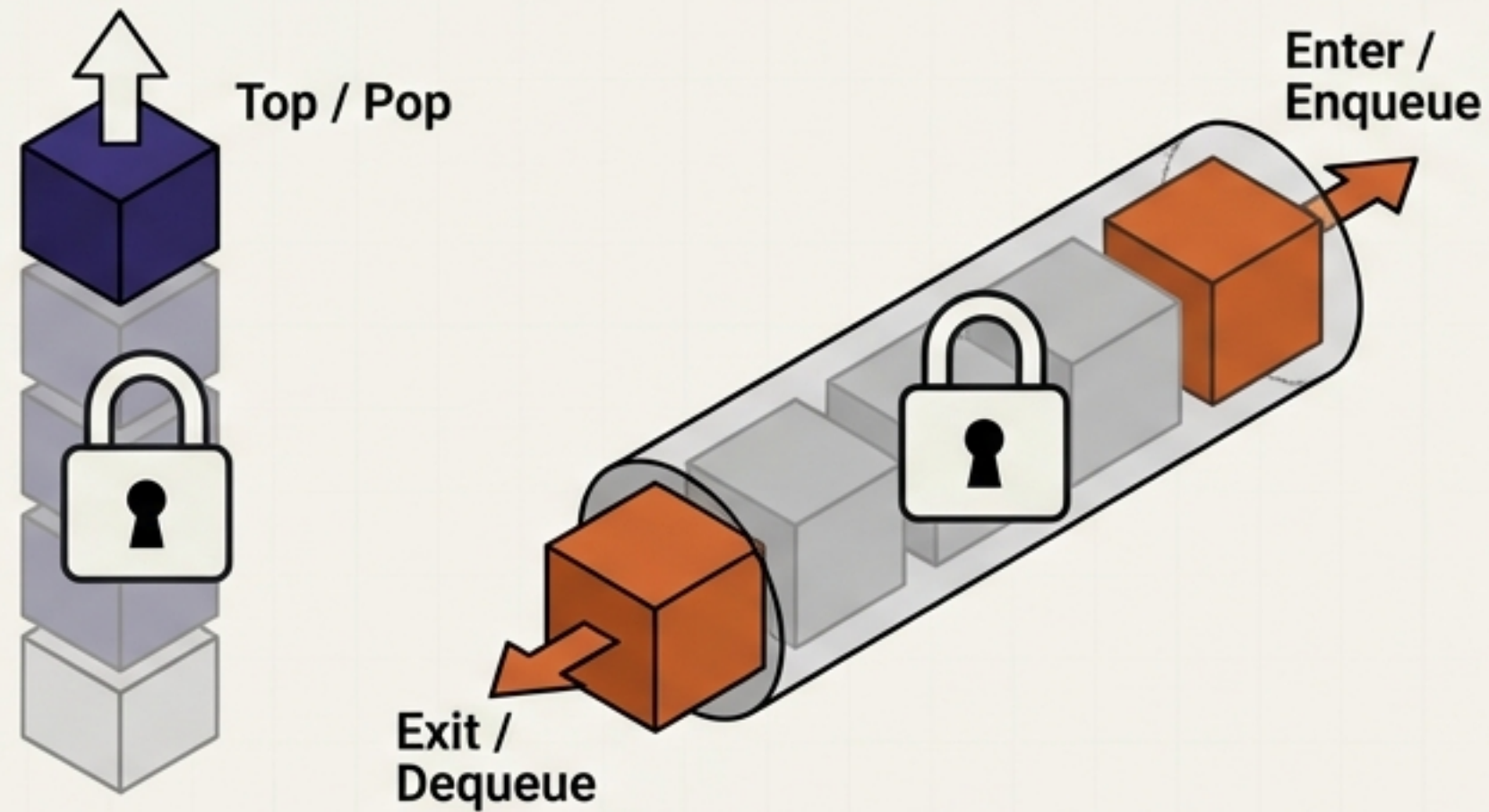
Time Complexity: $O(1)$ - Fast

```
from collections import deque
queue = deque()
queue.append(10) # Enqueue
queue.popleft() # Dequeue (Efficient!)
```


The Rules of Access: Can We Touch the Middle?

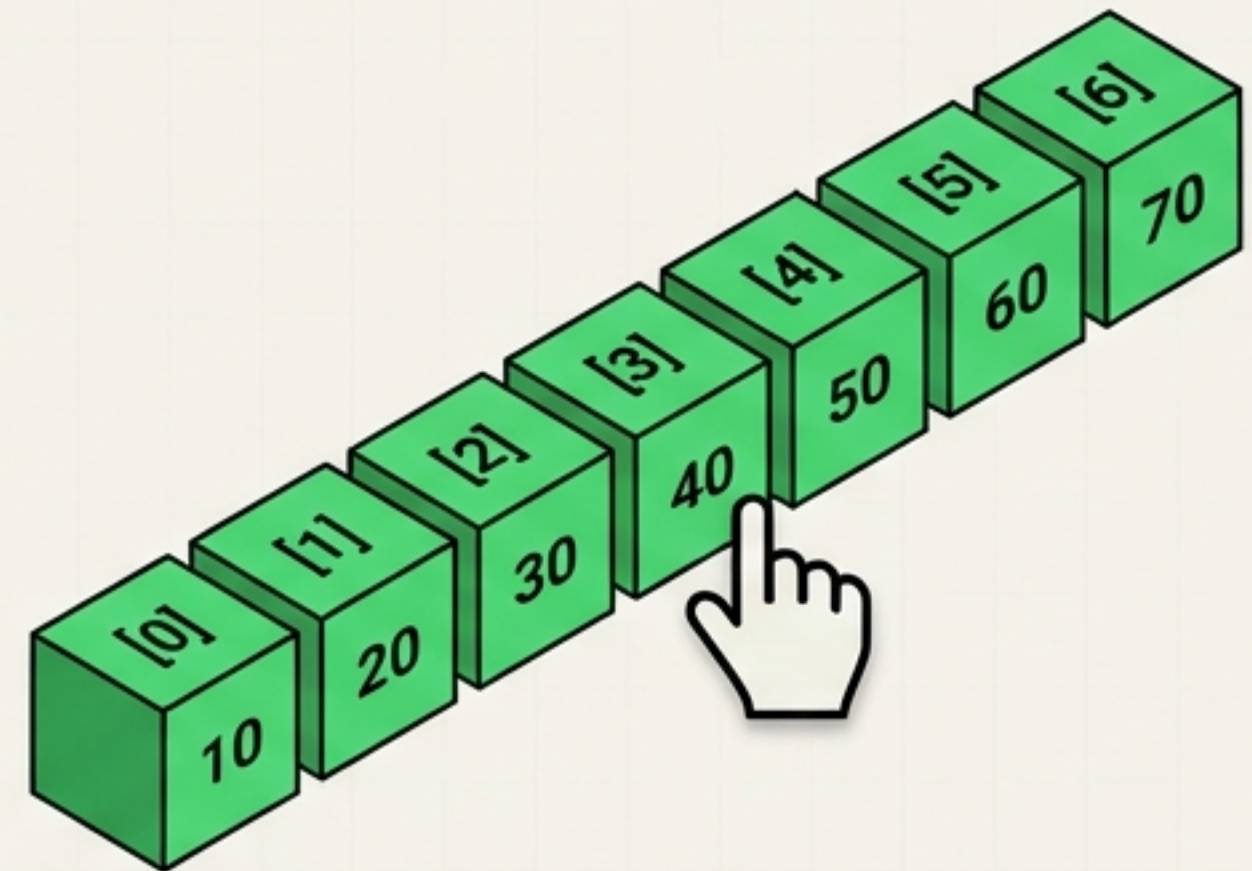
Data Structure Integrity Check

Image A (Stack/Queue)



Stack & Queue: NO Random Access.
Must remove predecessors to reach the middle.

Image B (Array/Linked List)



Array & Linked List: YES.
Random Access Allowed.

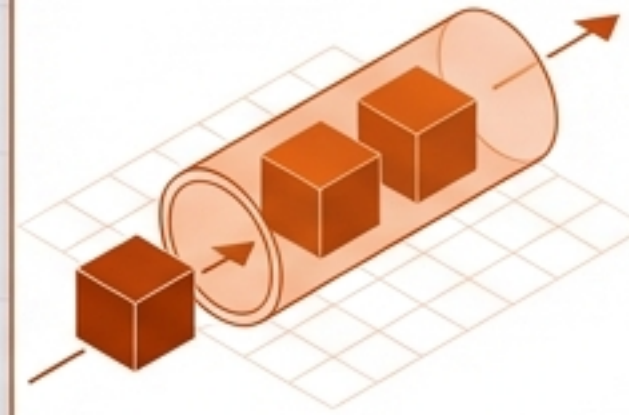
Verdict: If you need random access, a Stack/Queue is the wrong tool.

Bootcamp Recap & Key Takeaways



Stack (LIFO)

Best for undo mechanics and "Baseball Game" scoring.



Queue (FIFO)

Best for scheduling, buffering, and sequential processing.

Implementation

Use ``list`` for Stacks.

Use ``deque`` for Queues.

Always respect 32-bit constraints.

Mental Model

Defined not just by storage, but by strict rules of access.

Ready for the Next Challenge?



Mastery comes from practice. Try implementing these structures without looking at the solution code.

Content based on iCodeGuru Advanced DSA Bootcamp.